

untangle::actuator

Generated by Doxygen 1.18.0



|                                                               |    |
|---------------------------------------------------------------|----|
| 1 actuator                                                    | 1  |
| 2 Topic Index                                                 | 5  |
| 2.1 Topics                                                    | 5  |
| 3 Namespace Index                                             | 7  |
| 3.1 Namespace List                                            | 7  |
| 4 Concept Index                                               | 9  |
| 4.1 Concepts                                                  | 9  |
| 5 Class Index                                                 | 11 |
| 5.1 Class List                                                | 11 |
| 6 File Index                                                  | 13 |
| 6.1 File List                                                 | 13 |
| 7 Topic Documentation                                         | 15 |
| 7.1 namespace untangle: functions                             | 15 |
| 7.1.1 Detailed Description                                    | 15 |
| 7.1.2 Function Documentation                                  | 15 |
| 7.1.2.1 connect() [1/4]                                       | 15 |
| 7.1.2.2 connect() [2/4]                                       | 17 |
| 7.1.2.3 connect() [3/4]                                       | 17 |
| 7.1.2.4 connect() [4/4]                                       | 18 |
| 7.1.2.5 bind() [1/2]                                          | 19 |
| 7.1.2.6 bind() [2/2]                                          | 19 |
| 8 Namespace Documentation                                     | 21 |
| 8.1 untangle Namespace Reference                              | 21 |
| 8.1.1 Detailed Description                                    | 22 |
| 8.1.2 Typedef Documentation                                   | 22 |
| 8.1.2.1 task_t                                                | 22 |
| 8.1.3 Function Documentation                                  | 23 |
| 8.1.3.1 last_arg()                                            | 23 |
| 8.1.3.2 connect_one()                                         | 23 |
| 8.1.3.3 connect_named_one()                                   | 24 |
| 8.1.3.4 bind_task()                                           | 25 |
| 9 Concept Documentation                                       | 27 |
| 9.1 untangle::task_callback_for Concept Reference             | 27 |
| 9.1.1 Concept definition                                      | 27 |
| 9.1.2 Detailed Description                                    | 27 |
| 10 Class Documentation                                        | 29 |
| 10.1 untangle::actuator< action_t > Struct Template Reference | 29 |

|                                                |    |
|------------------------------------------------|----|
| 10.1.1 Detailed Description                    | 31 |
| 10.1.2 Member Typedef Documentation            | 32 |
| 10.1.2.1 actions_t                             | 32 |
| 10.1.2.2 tasks_t                               | 32 |
| 10.1.2.3 results_t                             | 32 |
| 10.1.2.4 errors_t                              | 33 |
| 10.1.3 Constructor & Destructor Documentation  | 33 |
| 10.1.3.1 actuator()                            | 33 |
| 10.1.4 Member Function Documentation           | 33 |
| 10.1.4.1 operator=()                           | 33 |
| 10.1.4.2 type()                                | 34 |
| 10.1.4.3 reset()                               | 34 |
| 10.1.4.4 copy_from()                           | 34 |
| 10.1.4.5 release_owned()                       | 35 |
| 10.1.4.6 invoke_callback()                     | 35 |
| 10.1.4.7 operator()()                          | 36 |
| 10.1.4.8 invoke_action()                       | 37 |
| 10.1.4.9 call_tasks()                          | 37 |
| 10.1.4.10 add() [1/4]                          | 38 |
| 10.1.4.11 add() [2/4]                          | 38 |
| 10.1.4.12 add() [3/4]                          | 39 |
| 10.1.4.13 add() [4/4]                          | 39 |
| 10.1.4.14 add_task()                           | 39 |
| 10.1.4.15 remove() [1/2]                       | 40 |
| 10.1.4.16 remove() [2/2]                       | 40 |
| 10.1.4.17 is_connected()                       | 41 |
| 10.1.4.18 has_tasks()                          | 41 |
| 10.1.4.19 has_action()                         | 41 |
| 10.1.5 Member Data Documentation               | 42 |
| 10.1.5.1 tasks                                 | 42 |
| 10.1.5.2 owned                                 | 42 |
| 10.2 untangle::invalid_action Struct Reference | 42 |
| 10.2.1 Detailed Description                    | 43 |
| 10.2.2 Constructor & Destructor Documentation  | 43 |
| 10.2.2.1 invalid_action()                      | 43 |
| 10.2.3 Member Function Documentation           | 43 |
| 10.2.3.1 what()                                | 43 |
| 11 File Documentation                          | 45 |
| 11.1 actuator.hpp                              | 45 |
| Index                                          | 51 |

# Chapter 1

## actuator

An actuator is a generic callable that may invoke a list of actions(callables of type `std::function<...>`).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. `async` implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

### Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

## Anonymous lambdas as actions

An actuator normally does not own its actions: it stores pointers to `std::function` objects the caller keeps alive. `add()` also has an overload that takes an action by value, so an anonymous lambda can be an action with no named variable to keep around, and returns a handle to pass to `remove()`:

```
untangle::actuator<std::function<int(int)>> actuator_scale;
auto* handle = actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add("triple", [](int v) { return v * 3; });
```

`connect()` accepts them too, but it deduces the action type from its arguments, and a lambda has its own closure type – it is not a `std::function` until something converts it. A call made only of anonymous lambdas therefore has nothing to deduce from, and the signature has to be named on `connect()` itself:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { /* ... */ }, [](int angle) { /* ... */ });
```

Naming the type of the variable the result is assigned to does not supply it – template arguments are deduced from the call arguments alone, never from what the returned value is assigned to:

```
// ill formed: "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { /* ... */ });
```

One named action anywhere in the call deduces the type for the whole of it, and the anonymous lambdas beside it then need nothing:

```
auto actuator_rotate = untangle::connect(action1, [](int angle) { /* ... */ });
```

The named form of `connect()` works the same way: a pair holding a pointer names an action the caller owns, a pair holding the action by value hands it over to the actuator, and the two can be mixed. The action type is deduced from the pointer, so a call whose first pair holds a lambda has to name the signature:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { /* ... */ }),
    std::make_pair("circle", [](int angle) { /* ... */ }));

// deduced from the leading pointer, so the lambda beside it needs nothing
auto actuator_mixed = untangle::connect(std::make_pair("triangle", &action1),
    std::make_pair("circle", [](int angle) { /* ... */ }));
```

## Tasks: an action bound to its arguments and to the callback it must notify

An action is a subscription: it lives across invocations, and it is fired with a pack the caller supplies each time. A task is a one-shot. It carries its own arguments, is fired once, and reports what it produced to a callback it was built with. The two live side by side in one actuator and share nothing else – `operator()` fires the actions and leaves the tasks alone, `call_tasks()` fires the tasks and leaves the actions alone.

A task exists so that work which runs later than it was described can still say what became of it. That is what a queue needs, and an actuator holding only actions cannot give: the callback convention on `operator()` reads the trailing argument of the invocation, and a queued call has no invocation left to read from.

```
untangle::actuator<std::function<int(int)>> actuator_scale;

// the callback comes last, after the arguments the action is bound to
actuator_scale.add_task(untangle::bind_task(action, 21, [](int result) { /* result == 42 */ }));

actuator_scale.call_tasks(); // runs it, notifies it, and empties the list
```

A task without a callback does not exist. That is the whole difference from an action's callback, which is optional and is recognised by its type – a trailing argument that does not look like a callback is simply an argument, and nothing fires. A task's callback is the last argument of `bind_task()`, taken by position, and `task_callback_for` only checks that it can serve. There is no answer "no" to fall through to, so a missing or unusable callback is a `static_assert` rather than silence.

It is not forwarded to the action either, so an action needs no callback parameter of its own – and a task whose action returns void is a task like any other. It reports with a void() callback, because finished is the message and the result is optional:

```
untangle::actuator<std::function<void(int)>> actuator_notify;
actuator_notify.add_task(untangle::bind_task(action, 5, [] { /* finished */ }));
```

The arguments are copied when the task is built, while the caller still holds them, and handed to the action as the task's own lvalues when it runs. A task outlives the call that described it and cannot borrow what may already be gone; a move-only argument therefore does not compile.

add\_task() answers bool. It refuses a task that has nothing to run or no callback to notify, and the refusal is the whole report – nothing is thrown and nothing is stored. Both are the caller's own mistake, and both are caught while the caller is still on the stack rather than surfacing later from call\_tasks() as a std::bad\_function\_call recorded against a task whose action had in fact succeeded.

Finished does not mean failed. A task whose action throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to actuator::errors, the tasks behind it still run, and a caller relying on the callback alone will never hear about it. A callback runs inside the same try as the task that owns it, so what it throws travels the same path – errors can hold a failure for a task that succeeded.

A task's result goes to its callback and nowhere else: actuator::results is how an action hands back what it returned, and a task was built with something better. call\_tasks() appends to errors rather than clearing it, while operator() clears both as it starts – so an actuator fired as one(); one.call\_tasks(); reports both kinds together, and the actions go first.

Ask has\_tasks() whether a task is waiting. is\_connected() answers for the actions and for nothing else: an actuator holding only tasks is not connected, and a queue reading the wrong one would report a batch of tasks as nothing to do.

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in doc/refman.pdf for further references.



## Chapter 2

# Topic Index

### 2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions . . . . . [15](#)



## Chapter 3

# Namespace Index

### 3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

|                          |                                                                   |                    |
|--------------------------|-------------------------------------------------------------------|--------------------|
| <a href="#">untangle</a> | Interface to <a href="#">untangle::actuator</a> functor . . . . . | <a href="#">21</a> |
|--------------------------|-------------------------------------------------------------------|--------------------|



## Chapter 4

# Concept Index

### 4.1 Concepts

Here is a list of all documented concepts with brief descriptions:

[untangle::task\\_callback\\_for](#)

Can `callback_t` serve as the completion callback of a task returning `result_t`? . . . . [27](#)



# Chapter 5

## Class Index

### 5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                                                                                 |                    |
|-----------------------------------------------------------------------------------------------------------------|--------------------|
| <a href="#">untangle::actuator&lt; action_t &gt;</a>                                                            |                    |
| An actuator is a functor that can trigger a dynamic list of actions (of type std::↵<br>function<...>) . . . . . | <a href="#">29</a> |
| <a href="#">untangle::invalid_action</a>                                                                        |                    |
| Invalid action exception . . . . .                                                                              | <a href="#">42</a> |



## Chapter 6

# File Index

### 6.1 File List

Here is a list of all documented files with brief descriptions:

|                                        |    |
|----------------------------------------|----|
| <a href="#">actuator.hpp</a> . . . . . | 45 |
|----------------------------------------|----|



# Chapter 7

## Topic Documentation

### 7.1 namespace untangle: functions

#### Functions

- `template<typename action_t, typename... Actions>`  
`actuator< action_t > untangle::connect (action_t &A1, Actions &&... An)`  
Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`  
`actuator< action_t > untangle::connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`  
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename key_t, typename... Actions>`  
`actuator< action_t > untangle::connect (std::pair< key_t, action_t * > A1, Actions &&... An)`  
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`  
requires (!std::is\_same\_v<value\_t, action\_t\*>)  
`actuator< action_t > untangle::connect (std::pair< key_t, value_t > A1, Actions &&... An)`  
Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵  
T>::type>>`  
`action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`  
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵  
T>::type>>`  
`action_t untangle::bind (class_t *obj, T class_t::*method)`  
Binding to a class method.

#### 7.1.1 Detailed Description

#### 7.1.2 Function Documentation

##### 7.1.2.1 connect() [1/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &&... An)
```

Creates an actuator holding an initial list of actions.

## Remarks

Ownership: an action passed as an lvalue is one the caller owns, and the actuator only points at it – it must outlive the actuator. An action passed as an rvalue, an anonymous lambda above all, is moved into the actuator instead and needs no named variable at all. The two can be mixed in one call. See the ownership convention on [actuator](#).

## Warning

A call made only of anonymous lambdas can not deduce `action_t`. A lambda has its own closure type and is not a `std::function` until something converts it, so there is nothing in such a call to deduce the action signature from, and it has to be named:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { ... }, [](int angle) { ... });
```

## Warning

Naming the type of the variable the result is assigned to does not supply it:

```
// still ill formed -- "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { ... });
```

Template arguments are deduced from the call arguments alone, never from what the returned value is assigned to, so the explicit argument belongs on [connect\(\)](#) itself. One named action anywhere in the call deduces `action_t` for the whole of it, and the anonymous lambdas beside it then need nothing.

## Parameters

|    |                                                                                |
|----|--------------------------------------------------------------------------------|
| A1 | - The first action. It is specified as <code>std::function&lt;...&gt;</code> . |
| An | - Any number of further actions, each convertible to the type of A1.           |

## Returns

An [actuator](#).

## Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle) {
    for (const auto& s : shapes) {
        s->rotate(angle);
    }
}

auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);

// connect() deduces the action type from its arguments. A lambda has its own closure type
// and is not a std::function until something converts it, so a call made only of anonymous
// lambdas has nothing to deduce from and the signature has to be named on connect()
// itself. Naming the type of the variable assigned to would not supply it: template
// arguments are never deduced from what the returned value is assigned to.
auto actuator_scale = untangle::connect<std::function<int(int)>>([](int v) { return v * 2; },
    [](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

7.1.2.2 `connect()` [2/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::type_identity_t< action_t > && A1,
    Actions &&... An)
```

Creates an actuator owning an initial list of actions.

This overload takes the first action by rvalue, so it is the one that accepts a leading anonymous lambda. `std::type_identity_t` makes `action_t` a non-deduced context here, which is what both keeps this overload out of the way of the lvalue one – it is not a candidate at all unless `action_t` is named – and states the requirement described by the warning on the overload taking the first action by lvalue.

## Parameters

|    |                                                                         |
|----|-------------------------------------------------------------------------|
| A1 | - The first action, as an rvalue.                                       |
| An | - Any number of further actions, owned or pointed at by value category. |

## Returns

An **actuator**.

7.1.2.3 `connect()` [3/4]

```
template<typename action_t, typename key_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::pair< key_t, action_t * > A1,
    Actions &&... An)
```

Creates an actuator holding an initial map of named actions.

## Remarks

Ownership works as it does for the unnamed **connect()** overloads: a pair holding a pointer names an action the caller owns, and a pair holding an action by value – an anonymous lambda above all – hands it to the actuator to own. The two can be mixed in one call.

## Warning

The action type is deduced from the pointer in the first pair, so a call whose first pair holds an anonymous lambda has nothing to deduce it from and has to name the signature on **connect()** itself, exactly as the unnamed overload does:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { ... })),
    std::make_pair("circle", [](int angle) { ... }));
```

## Remarks

`action_t` leads the template parameter list, ahead of `key_t`, so that naming it – **connect**<std::function<...>>(...) – means the same thing on every **connect()** overload. Both parameters are deduced from the first pair whenever it holds a pointer, so nothing has to be named here in the first place.

## Parameters

|    |                                                                                                                                     |
|----|-------------------------------------------------------------------------------------------------------------------------------------|
| A1 | - The first name/action pair. Its second element is a pointer to an action.                                                         |
| An | - Any number of further name/action pairs, owned or pointed at according to whether they hold the action itself or a pointer to it. |

## Returns

An [actuator](#).

## Example:

```
// A pair holding the action itself hands it to the actuator to own. The action type is
// deduced from a pointer, and there is no pointer here, so the signature has to be named
// on connect() -- as it does for the unnamed overload.
auto actuator_scale = untangle::connect<std::function<int(int)>>(
    std::make_pair("double", [](int v) { return v * 2; }),
    std::make_pair("triple", [](int v) { return v * 3; }));

actuator_scale.invoke_action("double", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20));
actuator_scale.invoke_action("triple", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(30));
```

## 7.1.2.4 connect() [4/4]

```
template<typename action_t, typename key_t, typename value_t, typename... Actions>
requires (!std::is_same_v<value_t, action_t*>)
actuator< action_t > untangle::connect (
    std::pair< key_t, value_t > A1,
    Actions &&... An)
```

Creates an actuator owning an initial map of named actions.

This overload takes a first pair holding the action itself rather than a pointer to it, so it is the one that accepts a leading anonymous lambda. `action_t` is never deduced here – the `requires` clause is what keeps it from competing with the overload taking a pointer in the first pair over a leading pointer pair, which both would otherwise accept once `action_t` is named.

## Parameters

|    |                                                          |
|----|----------------------------------------------------------|
| A1 | - The first name/action pair, holding the action itself. |
| An | - Any number of further name/action pairs.               |

## Returns

An [actuator](#).

## 7.1.2.5 bind() [1/2]

```
template<typename class__t, typename T, typename action__t = std::function<typename function_remove_const<T>::<
type>>
action__t untangle::bind (
    const std::shared_ptr< class__t > & obj,
    T class__t::* method)
```

Binding to a class function member.

It returns a `std::function(lambda)` that wraps the function member. It may be used to provide an action for [connect\(\)](#) or [actuator::add\(\)](#).

## Remarks

It requires a shared pointer to the class type. A `std::weak_ptr` to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an [actuator](#), and safe for the action to outlive the caller's shared pointer.

## Parameters

|        |                                                                                                                |
|--------|----------------------------------------------------------------------------------------------------------------|
| obj    | - Class object.                                                                                                |
| method | - Pointer to function member. It is specified as <code>&amp;&lt;class type&gt;::&lt;function member&gt;</code> |

## Returns

action\_\_t - A `std::function` that wraps the pointer to function member.

## Remarks

If the class object gets invalid, invoking this binding will throw an exception of type [invalid\\_action](#).

## 7.1.2.6 bind() [2/2]

```
template<typename class__t, typename T, typename action__t = std::function<typename function_remove_const<T>::<
type>>
action__t untangle::bind (
    class__t * obj,
    T class__t::* method)
```

Binding to a class method.

## Attention

It is not safe to use this binding when the pointed-to object may be destroyed. A null pointer is detected and reported as a dead action, but a pointer to an already destroyed object can not be distinguished from a valid one. Prefer the overload taking a `std::shared_ptr`.

## Remarks

It is provided for convenience of use: within a class it is safe to create bindings through this pointer.

Binding a null pointer produces an action that is always dead: invoking it throws [invalid\\_action](#).

## Parameters

|        |                                                                                                                |
|--------|----------------------------------------------------------------------------------------------------------------|
| obj    | - Pointer to class.                                                                                            |
| method | - Pointer to function member. It is specified as <code>&amp;&lt;class type&gt;::&lt;function member&gt;</code> |

## Returns

action\_t - A `std::function` that wraps the pointer to function member.

## Chapter 8

# Namespace Documentation

### 8.1 untangle Namespace Reference

Interface to `untangle::actuator` functor.

#### Classes

- struct `invalid_action`  
Invalid action exception.
- struct `actuator`  
An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

#### Concepts

- concept `task_callback_for`  
Can `callback_t` serve as the completion callback of a task returning `result_t`?

#### Typedefs

- using `task_t` = `std::function<void()>`  
An action, its arguments and the callback it notifies, sealed into one nullary callable.

#### Functions

- template<typename... Args>  
auto `last_arg` (Args &&... args)  
Get last argument of params pack;.
- template<typename action\_t, typename arg\_t>  
void `connect_one` (`actuator`< action\_t > &target, arg\_t &&arg)  
Add one `connect()` argument to the actuator being built.
- template<typename action\_t, typename... Actions>  
`actuator`< action\_t > `connect` (action\_t &A1, Actions &&... An)  
Creates an actuator holding an initial list of actions.

- `template<typename action_t, typename... Actions>`  
`actuator< action_t > connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`  
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename entry_t>`  
`void connect_named_one (actuator< action_t > &target, entry_t &&entry)`  
Add one named `connect()` argument to the actuator being built.
- `template<typename action_t, typename key_t, typename... Actions>`  
`actuator< action_t > connect (std::pair< key_t, action_t * > A1, Actions &&... An)`  
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`  
requires `(!std::is_same_v<value_t, action_t*>)`  
`actuator< action_t > connect (std::pair< key_t, value_t > A1, Actions &&... An)`  
Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`  
`T>::type>>`  
`action_t bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`  
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`  
`T>::type>>`  
`action_t bind (class_t *obj, T class_t::*method)`  
Binding to a class method.
- `template<typename action_t, typename... Args>`  
`task_t bind_task (action_t action, Args &&... args)`  
Binds an action to its arguments and to the callback it must notify, which comes last.

### 8.1.1 Detailed Description

Interface to `untangle::actuator` functor.

### 8.1.2 Typedef Documentation

#### 8.1.2.1 `task_t`

using `untangle::task_t = std::function<void()>`

An action, its arguments and the callback it notifies, sealed into one nullary callable.

What `bind_task()` builds, `actuator::add_task()` holds and `actuator::call_tasks()` fires.

#### Remarks

An action is a subscription; a task is a one-shot. An action is fired with a pack the caller supplies each time; a task carries its own arguments, is fired once, and reports to its own callback – so it is held by value, and nothing removes one because `call_tasks()` consumes it.

Running it is what reports. It returns nothing: the result goes to the callback and nowhere else, so there is nothing to read back and no part of it to reach from outside.

Neither the argument types nor the result type appear here, so one container holds tasks built from completely different actions – which is what lets a queue of them exist.

Anything that can be a `std::function<void()>` is a task; work already bound needs no `bind_task()`. What makes it a task is being fired once, and consumed.

### 8.1.3 Function Documentation

#### 8.1.3.1 last\_arg()

```
template<typename... Args>
auto untangle::last_arg (
    Args &&... args)
```

Get last argument of params pack;.

#### Template Parameters

|      |                       |
|------|-----------------------|
| Args | params template types |
|------|-----------------------|

#### Parameters

|      |        |
|------|--------|
| args | params |
|------|--------|

#### Returns

last param value in case it exist, otherwise 0.

#### 8.1.3.2 connect\_one()

```
template<typename action_t, typename arg_t>
void untangle::connect_one (
    actuator< action_t > & target,
    arg_t && arg)
```

Add one [connect\(\)](#) argument to the actuator being built.

An argument that is an lvalue of the action type is one the caller owns: only its address is stored, and it has to outlive the actuator. Anything else – an anonymous lambda, a temporary `std::function`, a named lambda that is not an `action_t` yet – has no owner to outlive the actuator, so it is converted to an action and moved into it.

#### Remarks

Empty actions are dropped rather than stored, which is what [connect\(\)](#) has always done with them. Dropping an empty owned action before it is stored is what keeps [actuator::owned](#) free of entries nothing points at.

## Template Parameters

|                              |                                                            |
|------------------------------|------------------------------------------------------------|
| <code>action_↔<br/>_t</code> | Action type of the actuator being built.                   |
| <code>arg_t</code>           | Deduced type of the argument, carrying its value category. |

## Parameters

|                     |                                           |
|---------------------|-------------------------------------------|
| <code>target</code> | - The actuator being built.               |
| <code>arg</code>    | - One <a href="#">connect()</a> argument. |

8.1.3.3 `connect_named_one()`

```
template<typename action_t, typename entry_t>
void untangle::connect_named_one (
    actuator< action_t > & target,
    entry_t && entry)
```

Add one named [connect\(\)](#) argument to the actuator being built.

It is [connect\\_one\(\)](#) for the named overloads: the second element of the pair decides ownership. A pointer to an action is one the caller owns and is only stored; anything else – an anonymous lambda, a temporary `std::function` – is converted to an action and moved into the actuator.

## Remarks

A null pointer is dropped rather than stored, which is what the named [connect\(\)](#) has always done with one. An owned action that is empty is dropped for the same reason the unnamed [connect\\_one\(\)](#) drops one: nothing would ever point at it.

## Template Parameters

|                              |                                                        |
|------------------------------|--------------------------------------------------------|
| <code>action_↔<br/>_t</code> | Action type of the actuator being built.               |
| <code>entry_t</code>         | Deduced type of the pair, carrying its value category. |

## Parameters

|                     |                             |
|---------------------|-----------------------------|
| <code>target</code> | - The actuator being built. |
| <code>entry</code>  | - One name/action pair.     |

8.1.3.4 `bind_task()`

```
template<typename action_t, typename... Args>
task_t untangle::bind_task (
    action_t action,
    Args &&... args)
```

Binds an action to its arguments and to the callback it must notify, which comes last.

Running the `task_t` it returns invokes the action and then hands the callback what it returned. A task runs later than it was built, so the arguments are copied here, while the caller still holds them, and handed to the action as the task's own lvalues.

## Remarks

The callback is the last argument, by position – not a trailing argument recognised by its type, as an action's is. A pack cannot be followed by a deducible parameter, so it arrives inside args and is split off here; every caller above just forwards (action, args...). `task_callback_for` only asks whether it can serve, which is what lets it be required.

It is not forwarded to the action, so an action needs no callback parameter of its own – and a void action, which could never have used one, is a task like any other.

## Attention

Each argument is copied, so a move-only one does not compile.

An empty callback – which the signature cannot rule out – or an empty action builds an empty task, which `actuator::add_task()` refuses. Caught here rather than thrown out of `actuator::call_tasks()` as a `std::bad_function_call`, where it would read as the failure of a task that had in fact run. Only a callable that can be tested for emptiness is tested, so a bare lambda is never refused.

## Template Parameters

|                       |                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>action_t</code> | Type of the action. Any callable naming a <code>result_type</code> will do; it need not be a <code>std::function</code> . |
| <code>Args</code>     | Types of the arguments to bind, of which the last is the callback.                                                        |

## Parameters

|                     |                                                                        |
|---------------------|------------------------------------------------------------------------|
| <code>action</code> | - The action to bind.                                                  |
| <code>args</code>   | - The arguments to bind to action, followed by the callback to notify. |

## Returns

The task, or an empty one – see the attention above.



## Chapter 9

# Concept Documentation

### 9.1 `untangle::task_callback_for` Concept Reference

Can `callback_t` serve as the completion callback of a task returning `result_t`?

```
#include <actuator.hpp>
```

#### 9.1.1 Concept definition

```
template<typename callback_t, typename result_t>
concept task_callback_for =
    requires(callback_t& c, result_t& r) { requires std::is_void_v<decltype(c(r))>; } ||
    (std::is_void_v<result_t> &&
     requires(callback_t& c) { requires std::is_void_v<decltype(c())>; })
```

#### 9.1.2 Detailed Description

Can `callback_t` serve as the completion callback of a task returning `result_t`?

A task's callback is required and is the last argument given to `bind_task()`, by position, so this asks only whether it can serve – not whether it was meant as one, which is what an action's callback convention asks. A task returning nothing still reports that it finished, so for `result_t` of `void` the callback takes no argument.

#### Attention

Both disjuncts are load-bearing. The first yields false for a void result rather than failing hard – `void&` in a `requires`-parameter list is a substitution failure in the immediate context – which leaves the second to decide; the second's `std::is_void_v` guard keeps a `std::function<void()>` from satisfying this for any result type, which would leave the result unreported.

#### Template Parameters

|                                           |                       |
|-------------------------------------------|-----------------------|
| <code>callback_</code><br><code>_t</code> | Type of the callback. |
|-------------------------------------------|-----------------------|

|          |                                        |
|----------|----------------------------------------|
| result_t | Result type of the task it reports on. |
|----------|----------------------------------------|

# Chapter 10

## Class Documentation

### 10.1 untangle::actuator< action\_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

```
#include <actuator.hpp>
```

#### Public Types

- using `actions_t` = `std::list<action_t*>`  
Actions container type.
- using `tasks_t` = `std::list<task_t>`  
Tasks container type.
- using `results_t` = `std::vector<typename result_t::type>`  
Results container type.
- using `errors_t` = `std::vector<std::exception_ptr>`  
Errors container type.

#### Public Member Functions

- `actuator` (const actuator &other)  
Copy constructor.
- actuator & `operator=` (const actuator &other)  
Assignment operator.
- action\_t `type` () const  
Helper method to access the action type of this object.
- void `reset` ()  
Remove all actions, and any stored results and errors.
- void `copy_from` (const actuator &other)  
Copy another actuator, re-pointing the copied actions at this object's storage.
- void `release_owned` (const action\_t \*action)  
Destroy an owned action that nothing points at any more.
- template<typename last\_t>  
void `invoke_callback` (last\_t &last)  
Invoke the trailing callback argument, if the invocation has one.

- `template<typename... Args>`  
`void operator() (Args &&... args)`  
The call operator.
- `template<typename... Args>`  
`void invoke_action (const std::string &name, Args &&... args)`  
Invokes one single action associated with a key.
- `void call_tasks ()`  
Fires every task held, in the order `add_task()` took them, and empties the list.
- `void add (action_t *action)`  
Add an action to the actions list.
- `action_t * add (action_t &&action)`  
Add an action the actuator owns.
- `void add (const std::string &name, action_t *action)`  
Add action to the actions map associated with a name.
- `action_t * add (const std::string &name, action_t &&action)`  
Add an action the actuator owns, associated with a name.
- `bool add_task (task_t &&task)`  
Adds a task, to be fired by `call_tasks()`.
- `void remove (const action_t *action)`  
Remove an action from the actions list.
- `void remove (const std::string &name)`  
Remove an action from actions map.
- `bool is_connected () const`  
Check if this actuator is "connected" with other actions.
- `bool has_tasks () const`  
Is there a task waiting to be fired?
- `bool has_action (const std::string &name) const`  
Check if there is certain named action.

## Public Attributes

- `actions_t` actions  
Actions list.
- `actions_map_t` actions\_map  
Named actions map.
- `results_t` results  
Actions return values list.
- `errors_t` errors  
What the actions threw. See `errors_t`.
- `tasks_t` tasks  
Tasks waiting to be fired, in the order `add_task()` took them.
- `std::list< action_t > owned`  
Actions this actuator owns, as added by `add(action_t&&)`.

### 10.1.1 Detailed Description

```
template<typename action_t>
struct untangle::actuator< action_t >
```

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

#### Remarks

An actuator object can be constructed with an initial list of actions by [connect\(\)](#).

Callback convention: when the last argument of an invocation can be called with the action return type `R` and returns nothing, it is not passed on as a plain argument alone – it is also treated as a completion callback. The action is invoked with the full argument list as usual, and the callback is then invoked with the action return value. Any void returning callable qualifies: a `std::function<void(R)>`, a lambda taking `R`, a function pointer. It applies to [operator\(\)](#) and to [invoke\\_action\(\)](#) alike. Actions returning void have no result to report, so no callback is invoked for them.

#### Warning

The return type is what separates a callback from an argument the action means to consume itself. A trailing callable that returns a value – a transform, a comparator – is left alone, and by the same rule a callback written to return something is silently not invoked. A callback returns nothing.

#### Remarks

Argument convention: an action must not take ownership of the arguments it is invoked with. [operator\(\)](#) forwards one argument pack to every action in the list, and an rvalue can be moved from only once, so an action that consumes an argument leaves the actions after it holding a moved-from object – an empty `std::function` for a callback a later action means to call itself. The arguments are forwarded rather than copied so that a single action pays nothing for the broadcast; what that buys has to be respected by the actions.

#### Warning

An action does not have to move an argument in its body to consume it: a parameter taken by value is move constructed from an rvalue by the call itself. Take arguments by reference or const reference, or invoke with lvalues, whenever more than one action is connected. The behaviour is otherwise unpredictable, and it is the caller and the action signatures together that decide it – an actuator cannot detect the violation.

#### Remarks

[invoke\\_action\(\)](#) invokes one single action, so nothing follows it and the convention does not constrain it.

Failure convention: an action that throws does not stop the invocation. What it threw is stored in [actuator::errors](#), one `std::exception_ptr` per failure, and the actions after it still run. An [untangle::invalid\\_action](#) means a dead binding, so that action is dropped as well; anything else leaves the action in place. Nothing is printed and nothing escapes the call - the caller reads [actuator::errors](#) and decides.

Ownership convention: an actuator normally does not own its actions. It stores pointers to `std::function` objects the caller keeps alive, and those have to outlive it. [add\(action\\_t&&\)](#) is the exception – it moves the action into [actuator::owned](#) and points at the stored copy – which is what lets [add\\_anonymous\\_lambda](#) be an action, with no named variable to keep around. The handle it returns is the only way to [remove\(\)](#) such an action afterwards.

## Template Parameters

|               |                                                                         |
|---------------|-------------------------------------------------------------------------|
| action↔<br>_t | Action type. It is specified as <code>std::function&lt;...&gt;</code> . |
|---------------|-------------------------------------------------------------------------|

## 10.1.2 Member Typedef Documentation

### 10.1.2.1 actions\_t

```
template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>
```

Actions container type.

#### Remarks

The elements stored are of pointer type, that is required to implement the [remove\(\)](#) operation. `std::function` supports only equality operator for nullptr (two `std::function(s)` can not compare).

### 10.1.2.2 tasks\_t

```
template<typename action_t>
using untangle::actuator< action_t >::tasks_t = std::list<task\_t>
```

Tasks container type.

#### Remarks

The elements are values, not pointers: identity is what [remove\(\)](#) needs of an action, and no one removes a single task – [call\\_tasks\(\)](#) consumes the whole list.

A `std::list` for the same reason [owned](#) is one: adding never invalidates the address of an element already in it, so a reference taken into the list stays good.

It names no result type, because [untangle::task\\_t](#) does not.

### 10.1.2.3 results\_t

```
template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>
```

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from [results](#).

#### 10.1.2.4 errors\_t

```
template<typename action_t>
using untangle::actuator< action_t >::errors_t = std::vector<std::exception_ptr>
```

Errors container type.

It holds what the actions threw, as `std::exception_ptr`, in the order they were invoked.

### 10.1.3 Constructor & Destructor Documentation

#### 10.1.3.1 actuator()

```
template<typename action_t>
untangle::actuator< action_t >::actuator (
    const actuator< action_t > & other) [inline]
```

Copy constructor.

Remarks

It can not be defaulted, see [copy\\_from\(\)](#).

### 10.1.4 Member Function Documentation

#### 10.1.4.1 operator=()

```
template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
    const actuator< action_t > & other) [inline]
```

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

#### 10.1.4.2 type()

```
template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]
```

Helper method to access the action type of this object.

Note

Intended to be used in expressions by decltype(<[actuator](#) instance>.[type\(\)](#))

Returns

`action_t`

#### 10.1.4.3 reset()

```
template<typename action_t>
void untangle::actuator< action_t >::reset () [inline]
```

Remove all actions, and any stored results and errors.

After this call the actuator is empty: [actuator::is\\_connected\(\)](#) returns false. The actions it owns are destroyed, so every handle returned by [add\(action\\_t&&\)](#) is dangling afterwards; the actions it merely points at are left untouched.

#### 10.1.4.4 copy\_from()

```
template<typename action_t>
void untangle::actuator< action_t >::copy_from (
    const actuator< action_t > & other) [inline]
```

Copy another actuator, re-pointing the copied actions at this object's storage.

Remarks

A defaulted copy is wrong as soon as an actuator owns anything: it duplicates [actuator::owned](#) but leaves [actuator::actions](#) pointing into other's copy of it, so the new object dangles the moment other is destroyed. The owned actions are copied first, then each pointer into other::owned is translated to the matching element here.

A pointer to an action the caller owns is external to both actuators and is copied unchanged: it is not in either [actuator::owned](#), so it is not in the translation table and is left as it is.

Parameters

|       |                                                               |
|-------|---------------------------------------------------------------|
| other | - The actuator to copy. Self copy is the caller's to exclude. |
|-------|---------------------------------------------------------------|

## 10.1.4.5 release\_owned()

```
template<typename action_t>
void untangle::actuator< action_t >::release_owned (
    const action_t * action) [inline]
```

Destroy an owned action that nothing points at any more.

## Remarks

Removing an action only drops a pointer to it. When the actuator owns that action the std::function itself lives in `actuator::owned` and has to go too, or a loop of `add(action_t&&)` and `remove()` grows `actuator::owned` without bound.

The action is kept while anything still refers to it: one handle can be added to both the list and the map, and removing it from one must not leave the other pointing at a destroyed object. An action the actuator does not own is not in `actuator::owned`, so nothing is found to erase and the caller's object is left alone.

## Parameters

|        |                                                                            |
|--------|----------------------------------------------------------------------------|
| action | - The action just removed. A null pointer is accepted and matches nothing. |
|--------|----------------------------------------------------------------------------|

## 10.1.4.6 invoke\_callback()

```
template<typename action_t>
template<typename last_t>
void untangle::actuator< action_t >::invoke_callback (
    last_t & last) [inline]
```

Invoke the trailing callback argument, if the invocation has one.

Does nothing unless last\_t can be called with the action return type and returns nothing, so any other trailing argument is left alone. The test is on what the type can do, not on what it is, so a plain lambda qualifies as well as a std::function. See the callback convention on `actuator`.

## Remarks

The single condition does the work of two tests: (1) whether last can be called with the action return type at all, and (2) whether that call returns nothing. Spelled out as traits they need two nested if constexpr:

```
// (1)
if constexpr (std::is_invocable_v<last_t&, typename result_t::type>) {
    // (2)
    if constexpr (std::is_void_v<std::invoke_result_t<last_t&, typename result_t::type>)>) {
        last(results.back());
    }
}
```

Both are folded into the one expression instead, each in a different part of it:

```
requires { requires std::is_void_v<decltype( last(results.back()) )>; }
// \underline{(2)} \underline{(1)}
```

## Remarks

(1) Invocability comes for free. To form `decltype(last(results.back()))` the compiler must first form the call `last(...)`. If `last` is an int, that expression does not exist – and inside a `requires-expression` a substitution failure yields `false` instead of being ill formed, the same property concepts are built on. So invocability is never tested separately: it is a precondition of the return type question, and failing it is what makes the whole condition false.

(2) The return type test is the inner `requires`, and the doubled keyword is not a typo. `requires { expr; }` asks only whether `expr` is well formed, and `std::is_void_v<...>` is well formed when it is `false` too – that spelling would let a value returning callable through. The second keyword makes it a nested requirement: a condition that has to hold, not merely compile.

The trait spelling has to nest, which is part of why it is not the one used here: the two `if constexpr` cannot be joined with `&&`. `&&` short circuits evaluation, not instantiation, so `std::invoke_result_t` is formed whatever the first trait says, and for a type that is not invocable at all it has no `::type` – a hard error rather than a false result.

The caller must copy the callback out of the argument pack before invoking the action: an action taking it by value moves from the caller's `std::function`, which would leave this an empty function to call.

## Template Parameters

|                                       |                                                                     |
|---------------------------------------|---------------------------------------------------------------------|
| <code>last_</code><br><code>_t</code> | Type of the last argument, as returned by <code>last_arg()</code> . |
|---------------------------------------|---------------------------------------------------------------------|

## Parameters

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| <code>last</code> | - Last argument value, copied before the action was invoked. |
|-------------------|--------------------------------------------------------------|

10.1.4.7 `operator>()()`

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::operator() (
    Args &&... args) [inline]
```

The call operator.

Actions in the `actuator::actions` list are triggered by invoking the call operator.

## Parameters

|                   |                                                                                                                                                                              |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>args</code> | - Arguments list must match the action arity. A trailing callback is invoked once per action, with that action's return value; see the convention on <code>actuator</code> . |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Warning

The same argument pack is forwarded to every action in the list, so the argument convention on [actuator](#) applies here in full: an action that consumes an argument leaves the actions after it with a moved-from object.

## Remarks

Nothing escapes this call: what an action throws goes to [actuator::errors](#), and the actions after it still run. See the failure convention on [actuator](#).

## 10.1.4.8 invoke\_action()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::invoke_action (
    const std::string & name,
    Args &&... args) [inline]
```

Invokes one single action associated with a key.

An action that can not be invoked – a null pointer, an empty std::function, a binding whose object is gone – is dropped from [actuator::actions\\_map](#) instead, exactly as [operator\(\)\(\)](#) drops one from [actuator::actions](#). Nothing is invoked and nothing escapes; [actuator::has\\_action\(\)](#) reports it gone afterwards.

## Remarks

What the action throws is stored in [actuator::errors](#), as it is for [operator\(\)\(\)](#). See the failure convention on [actuator](#).

## Parameters

|      |                                                                                                                                                               |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| name | - Key associated with the action. Invoking a key that is not in the map does nothing.                                                                         |
| args | - Arguments list must match the action arity. A trailing callback is invoked with the action's return value; see the convention on <a href="#">actuator</a> . |

## 10.1.4.9 call\_tasks()

```
template<typename action_t>
void untangle::actuator< action_t >::call_tasks () [inline]
```

Fires every task held, in the order [add\\_task\(\)](#) took them, and empties the list.

Each task notifies its own callback as it runs. It does not fire the actions, as [operator\(\)\(\)](#) does not fire the tasks, and a task's result goes to its callback alone – never to [actuator::results](#).

## Remarks

Errors are appended to [actuator::errors](#), not written over it. [operator\(\)\(\)](#) clears that list and this does not, so [one\(\)](#); [one.call\\_tasks\(\)](#); reports both kinds together – which means the actions go first.

## Attention

Finished does not mean failed. A task that throws does not stop the pass and is not notified; a callback that throws lands in [actuator::errors](#) the same way. The task is consumed either way, so a caller waiting on the callback alone must read the errors.

10.1.4.10 `add()` [1/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    action_t * action) [inline]
```

Add an action to the actions list.

## Parameters

|        |                       |
|--------|-----------------------|
| action | - Action to be added. |
|--------|-----------------------|

## Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.11 `add()` [2/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    action_t && action) [inline]
```

Add an action the actuator owns.

The action is moved into `actuator::owned`, so it needs no named variable outliving the actuator: an anonymous lambda can be passed straight in, and the call converts it to `action_t`. See the ownership convention on `actuator`.

## Parameters

|        |                                                            |
|--------|------------------------------------------------------------|
| action | - Action to be added. It is taken by value and moved from. |
|--------|------------------------------------------------------------|

## Returns

`action_t*` - A handle to the stored action, to pass to `remove()`. It stays valid until that action is removed or the actuator is destroyed. A copy of the actuator owns its own copy of the action and has its own handle to it.

## Example:

```
// An anonymous lambda has no named variable to outlive the actuator, so the actuator takes
// ownership of it: it is moved into actuator::owned and the list points at the stored copy.
untangle::actuator<std::function<int(int)>> actuator_scale;
actuator_scale.add([&](int v) { return v * 2; });
actuator_scale.add([&](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

## 10.1.4.12 add() [3/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    const std::string & name,
    action_t * action) [inline]
```

Add action to the actions map associated with a name.

## Parameters

|        |                       |
|--------|-----------------------|
| name   | - Name of the action. |
| action | - Action to be added. |

## 10.1.4.13 add() [4/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    const std::string & name,
    action_t && action) [inline]
```

Add an action the actuator owns, associated with a name.

## Parameters

|        |                                                            |
|--------|------------------------------------------------------------|
| name   | - Name of the action.                                      |
| action | - Action to be added. It is taken by value and moved from. |

## Returns

action\_t\* - A handle to the stored action, as `add(action_t&&)` returns, or nullptr when name is already taken. The name already in the map keeps the action it has, as it does for `add(const std::string&, action_t*)`, and the action offered here is dropped rather than left ownerless in `actuator::owned`.

## 10.1.4.14 add\_task()

```
template<typename action_t>
bool untangle::actuator< action_t >::add_task (
    task_t && task) [inline]
```

Adds a task, to be fired by `call_tasks()`.

It is stored by value: an actuator owns every task it holds, unlike the actions it merely points at. Anything that can be an `untangle::task_t` is taken, from `bind_task()` or not.

There is no counterpart to `remove()` and no named form: `call_tasks()` consumes the list, and tasks are fired as a batch in the order they were added.

## Parameters

|      |                                                                                   |
|------|-----------------------------------------------------------------------------------|
| task | - The task to add. It is taken by value and moved from; a refused one is dropped. |
|------|-----------------------------------------------------------------------------------|

## Returns

true - taken, and [call\\_tasks\(\)](#) will fire it.

false - refused: it is empty, so it can neither run nor report.

10.1.4.15 `remove()` [1/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const action_t * action)    [inline]
```

Remove an action from the actions list.

An invalid action (empty `std::function`) is implicitly removed when [operator\(\)\(\)](#) is invoked.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See [release\\_owned\(\)](#).

## Parameters

|        |                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------|
| action | - Action to be removed. For an owned action this is the handle returned by <a href="#">add(action_t&amp;&amp;)</a> . |
|--------|----------------------------------------------------------------------------------------------------------------------|

## Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.16 `remove()` [2/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const std::string & name)    [inline]
```

Remove an action from actions map.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See [release\\_owned\(\)](#).

## Parameters

|      |                                 |
|------|---------------------------------|
| name | - Name of the action to remove. |
|------|---------------------------------|

## 10.1.4.17 is\_connected()

```
template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]
```

Check if this actuator is "connected" with other actions.

## Remarks

It answers for the actions and for nothing else. Tasks are not actions and do not make an actuator connected; ask [has\\_tasks\(\)](#) about those. Kept this way deliberately when tasks arrived, because a caller that will never hold a task – an attachment, a poll – reads this and must keep reading the answer it always read.

## Returns

true - if the [actuator::actions](#) list is not empty.  
false - if the [actuator::actions](#) list is empty.

## 10.1.4.18 has\_tasks()

```
template<typename action_t>
bool untangle::actuator< action_t >::has_tasks () const [inline]
```

Is there a task waiting to be fired?

The counterpart to [is\\_connected\(\)](#), and the two answer different questions: this one is about [actuator::tasks](#), that one about [actuator::actions](#). An actuator can hold either, both, or neither.

## Remarks

It is what a queue asks to decide whether its worker still has work. Reading [is\\_connected\(\)](#) for that would report a batch of tasks as nothing to do, and the work would sit unfired.

A refused task is not a waiting one: [add\\_task\(\)](#) answering false means the task was dropped, so this stays false.

## Returns

true - a task is waiting; [call\\_tasks\(\)](#) has something to fire.  
false - no task is waiting. True again after [call\\_tasks\(\)](#), which consumes them.

## 10.1.4.19 has\_action()

---

```
Generated by Doxygen
template<typename action_t>
bool untangle::actuator< action_t >::has_action (
    const std::string & name) const [inline]
```

## Parameters

|      |                                    |
|------|------------------------------------|
| name | - Name associated with the action. |
|------|------------------------------------|

## Returns

true - if name can be found in [actuator::actions\\_map](#)  
false - if name can not be found in [actuator::actions\\_map](#)

## 10.1.5 Member Data Documentation

## 10.1.5.1 tasks

```
template<typename action_t>
tasks_t untangle::actuator< action_t >::tasks
```

Tasks waiting to be fired, in the order [add\\_task\(\)](#) took them.

## Remarks

Held beside [actions](#) and sharing nothing with them: [operator>\(\)](#) fires the actions and leaves these alone, [call\\_tasks\(\)](#) fires these and leaves the actions alone.

## 10.1.5.2 owned

```
template<typename action_t>
std::list<action_t> untangle::actuator< action_t >::owned
```

Actions this actuator owns, as added by [add\(action\\_t&&\)](#).

## Remarks

It is a `std::list` because inserting into one never invalidates the address of an element already in it, so every handle already handed out stays valid as further actions are added. A `std::vector` would reallocate and dangle all of them at once.

The documentation for this struct was generated from the following file:

- [actuator.hpp](#)

## 10.2 untangle::invalid\_action Struct Reference

Invalid action exception.

```
#include <actuator.hpp>
```

## Public Member Functions

- [invalid\\_action](#) (std::string text)  
Construct a new invalid action object.
- const char \* [what](#) () const noexcept override  
The message text describing the reason of this exception.

## 10.2.1 Detailed Description

Invalid action exception.

## Remarks

An action may be provided as a binding to a class function member, by using [untangle::bind\(\)](#).  
When the class object gets invalid, invoking the action will raise an exception to this type.

## 10.2.2 Constructor &amp; Destructor Documentation

## 10.2.2.1 invalid\_action()

```
untangle::invalid_action::invalid_action (
    std::string text) [inline], [explicit]
```

Construct a new invalid action object.

## Parameters

|      |                                                            |
|------|------------------------------------------------------------|
| text | - A message text, describing the reason of this exception. |
|------|------------------------------------------------------------|

## 10.2.3 Member Function Documentation

## 10.2.3.1 what()

```
const char * untangle::invalid_action::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

## Returns

const char\* - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp



# Chapter 11

## File Documentation

### 11.1 actuator.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <cstdint>
00009 #include <exception>
00010 #include <functional>
00011 #include <list>
00012 #include <map>
00013 #include <memory>
00014 #include <string>
00015 #include <tuple>
00016 #include <type_traits>
00017 #include <utility>
00018 #include <vector>
00019
00020 namespace untangle {
00021 // exception
00030 struct invalid_action : std::exception {
00036     explicit invalid_action(std::string text) : message(std::move(text)) {}
00037
00043     const char* what() const noexcept override { return message.c_str(); }
00044
00045 private:
00046     std::string message;
00047 };
00048
00056 template <typename... Args>
00057 auto last_arg(Args&&... args) {
00058     if constexpr (sizeof...(Args) <= 0) {
00059         return 0;
00060     } else {
00061         return std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00062     }
00063 }
00064
00082 template <typename callback_t, typename result_t>
00083 concept task_callback_for =
00084     requires(callback_t& c, result_t& r) { requires std::is_void_v<decltype(c(r))>; } ||
00085     (std::is_void_v<result_t> &&
00086      requires(callback_t& c) { requires std::is_void_v<decltype(c())>; });
00087
00106 using task_t = std::function<void()>;
00107
00159 template <typename action_t>
00160 struct actuator final {
00168     using actions_t = std::list<action_t*>;
00169     using actions_map_t = std::map<std::string, action_t*>;
00181     using tasks_t = std::list<task_t>;
00182     using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int,
00183                                     typename action_t::result_type>;
00190     using results_t = std::vector<typename result_t::type>;
00196     using errors_t = std::vector<std::exception_ptr>;
00197
00198     actions_t actions;
00199     actions_map_t actions_map;
00200     results_t results;
```

```

00201 errors_t errors;
00202
00209 tasks_t tasks;
00210
00218 std::list<action_t> owned;
00219
00220 actuator() = default;
00226 actuator(const actuator& other) { copy_from(other); }
00227 actuator(actuator&&) noexcept = default;
00228 ~actuator() = default;
00235 actuator& operator=(const actuator& other) {
00236     if (this != &other) {
00237         copy_from(other);
00238     }
00239     return *this;
00240 }
00241 actuator& operator=(actuator&&) noexcept = default;
00242
00250 action_t type() const { return nullptr; }
00251
00259 void reset() {
00260     actions.clear();
00261     actions_map.clear();
00262     results.clear();
00263     errors.clear();
00264     owned.clear();
00265 }
00266
00281 void copy_from(const actuator& other) {
00282     owned = other.owned;
00283
00284     // Old address -> new address, for the owned actions only.
00285     std::map<const action_t*, action_t*> remap;
00286     auto src = other.owned.begin();
00287     auto dst = owned.begin();
00288     for (; src != other.owned.end(); ++src, ++dst) {
00289         remap.emplace(&*src, &*dst);
00290     }
00291     const auto translate = [&remap](action_t* action) {
00292         const auto it = remap.find(action);
00293         return it != remap.end() ? it->second : action;
00294     };
00295
00296     actions.clear();
00297     for (const auto& action : other.actions) {
00298         actions.push_back(translate(action));
00299     }
00300     actions_map.clear();
00301     for (const auto& entry : other.actions_map) {
00302         actions_map.emplace(entry.first, translate(entry.second));
00303     }
00304     // Named explicitly, like every member here: this function does not default, so anything it
00305     // does not copy is silently lost by every copy of an actuator.
00306     tasks = other.tasks;
00307     results = other.results;
00308     errors = other.errors;
00309 }
00310
00325 void release_owned(const action_t* action) {
00326     for (const auto& a : actions) {
00327         if (a == action) {
00328             return;
00329         }
00330     }
00331     for (const auto& entry : actions_map) {
00332         if (entry.second == action) {
00333             return;
00334         }
00335     }
00336     owned.remove_if([action](const action_t& a) { return &a == action; });
00337 }
00338
00394 template <typename last_t>
00395 void invoke_callback(last_t& last) {
00396     if constexpr (requires { requires std::is_void_v<decltype(last(results.back()))>; }) {
00397         last(results.back());
00398     }
00399 }
00400
00416 template <typename... Args>
00417 void operator()(Args&&... args) {
00418     results.clear();
00419     errors.clear();
00420
00421     // Copied up front, before any action can move the callback out of the argument pack.
00422     auto last = last_arg(args...);
00423

```

```

00424 // Dead bindings are collected here and dropped after the loop.
00425 // The actuator does not own the actions it points at, so it must never
00426 // write through action_t* into a std::function belonging to the caller.
00427 std::vector<action_t*> dead_actions;
00428
00429 for (const auto& action : actions) {
00430     // A null pointer or an empty std::function can never be invoked. Calling an
00431     // empty one throws std::bad_function_call, which is not an invalid_action and
00432     // would escape this operator, so drop it instead of invoking it.
00433     if (action == nullptr || !*action) {
00434         dead_actions.push_back(action);
00435         continue;
00436     }
00437
00438     try {
00439         if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00440             (*action)(std::forward<Args>(args)...);
00441         } else {
00442             results.push_back((*action)(std::forward<Args>(args)...));
00443             invoke_callback(last);
00444         }
00445     } catch (const invalid_action&) {
00446         // A dead binding is not coming back: it is recorded and the action is dropped.
00447         errors.push_back(std::current_exception());
00448         dead_actions.push_back(action);
00449     } catch (...) {
00450         // Anything else is the action's own failure. It is recorded, the action is kept, and the
00451         // actions behind it still run.
00452         errors.push_back(std::current_exception());
00453     }
00454 }
00455
00456 for (const auto& dead_action : dead_actions) {
00457     actions.remove(dead_action);
00458     release_owned(dead_action);
00459 }
00460 }
00461
00478 template <typename... Args>
00479 void invoke_action(const std::string& name, Args&&... args) {
00480     results.clear();
00481     errors.clear();
00482
00483     // Copied up front, before the action can move the callback out of the argument pack.
00484     auto last = last_arg(args...);
00485     const auto it = actions_map.find(name);
00486     if (it == actions_map.end()) {
00487         return;
00488     }
00489
00490     // A null pointer or an empty std::function can never be invoked, and dereferencing a
00491     // null one to try is undefined behaviour before a call is even made. Calling an empty
00492     // one throws std::bad_function_call, which is not an invalid_action and would escape
00493     // this method. Drop it instead, as operator()() does for the actions list.
00494     if (it->second == nullptr || !*it->second) {
00495         const action_t* dead_action = it->second;
00496         actions_map.erase(it);
00497         release_owned(dead_action);
00498         return;
00499     }
00500
00501     try {
00502         if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00503             (*it->second)(std::forward<Args>(args)...);
00504         } else {
00505             results.push_back((*it->second)(std::forward<Args>(args)...));
00506             invoke_callback(last);
00507         }
00508     } catch (const invalid_action&) {
00509         errors.push_back(std::current_exception());
00510         const action_t* dead_action = it->second;
00511         actions_map.erase(name);
00512         release_owned(dead_action);
00513     } catch (...) {
00514         errors.push_back(std::current_exception());
00515     }
00516 }
00517
00533 void call_tasks() {
00534     // Taken by swap, so a callback that adds a task adds it to the *next* pass. Iterating the
00535     // member instead would let a task that re-adds itself keep this loop from ever ending -- and
00536     // it is the same reason a queue built on this takes its batch by move.
00537     tasks_t firing;
00538     firing.swap(tasks);
00539
00540     for (auto& task : firing) {
00541         try {

```

```

00542     // The task notifies its own callback: bind_task() sealed the two together.
00543     task();
00544 } catch (...) {
00545     // The task's own failure, or its callback's -- the two are not told apart here, and the
00546     // attention above says why.
00547     errors.push_back(std::current_exception());
00548 }
00549 }
00550 }
00551
00560 void add(action_t* action) { actions.push_back(action); }
00561
00577 action_t* add(action_t&& action) {
00578     owned.push_back(std::move(action));
00579     actions.push_back(&owned.back());
00580     return &owned.back();
00581 }
00582
00589 void add(const std::string& name, action_t* action) { actions_map.emplace(name, action); }
00590
00601 action_t* add(const std::string& name, action_t&& action) {
00602     owned.push_back(std::move(action));
00603     const auto [it, inserted] = actions_map.emplace(name, &owned.back());
00604     if (!inserted) {
00605         owned.pop_back();
00606         return nullptr;
00607     }
00608     return &owned.back();
00609 }
00610
00630 bool add_task(task_t&& task) {
00631     if (!task) {
00632         return false;
00633     }
00634     tasks.push_back(std::move(task));
00635     return true;
00636 }
00637
00638 void remove(const action_t* action) {
00639     actions.remove_if([&action](const auto& a) { return (action == a); });
00640     release_owned(action);
00641 }
00642
00666 void remove(const std::string& name) {
00667     const auto it = actions_map.find(name);
00668     if (it == actions_map.end()) {
00669         return;
00670     }
00671     const action_t* action = it->second;
00672     actions_map.erase(it);
00673     release_owned(action);
00674 }
00675
00687 bool is_connected() const { return !actions.empty() || !actions_map.empty(); }
00688
00706 bool has_tasks() const { return !tasks.empty(); }
00707
00715 bool has_action(const std::string& name) const {
00716     return actions_map.find(name) != actions_map.end();
00717 }
00718 };
00719
00737 template <typename action_t, typename arg_t>
00738 void connect_one(actuator<action_t>& target, arg_t&& arg) {
00739     if constexpr (std::is_lvalue_reference_v<arg_t&&> &&
00740         std::is_same_v<std::decay_t<arg_t>, action_t>) {
00741         if (arg != nullptr) {
00742             target.add(&arg);
00743         }
00744     } else {
00745         action_t action(std::forward<arg_t>(arg));
00746         if (action != nullptr) {
00747             target.add(std::move(action));
00748         }
00749     }
00750 }
00751
00794 template <typename action_t, typename... Actions>
00795 actuator<action_t> connect(action_t& A1, Actions&&... An) {
00796     actuator<action_t> target;
00797     connect_one(target, A1);
00798     (connect_one(target, std::forward<Actions>(An)), ...);
00799     return target;
00800 }
00801
00818 template <typename action_t, typename... Actions>

```

```

00819 actuator<action_t> connect(std::type_identity_t<action_t>&& A1, Actions&&... An) {
00820     actuator<action_t> target;
00821     connect_one(target, std::move(A1));
00822     (connect_one(target, std::forward<Actions>(An)), ...);
00823     return target;
00824 }
00825
00826 template <typename actuator_t>
00827 void remove_empty_actions(actuator_t& actuator) {
00828     std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
00829     for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it) {
00830         if (it->second == nullptr) {
00831             removable_iterators.push_back(it);
00832         }
00833     }
00834     for (auto it : removable_iterators) {
00835         actuator.actions_map.erase(it);
00836     }
00837 }
00838
00856 template <typename action_t, typename entry_t>
00857 void connect_named_one(actuator<action_t>& target, entry_t&& entry) {
00858     if constexpr (std::is_same_v<std::remove_cvref_t<decltype(entry.second)>, action_t*>) {
00859         if (entry.second != nullptr) {
00860             target.add(entry.first, entry.second);
00861         }
00862     } else {
00863         action_t action(std::forward<entry_t>(entry).second);
00864         if (action != nullptr) {
00865             target.add(entry.first, std::move(action));
00866         }
00867     }
00868 }
00869
00904 template <typename action_t, typename key_t, typename... Actions>
00905 actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions&&... An) {
00906     actuator<action_t> target;
00907     connect_named_one(target, std::move(A1));
00908     (connect_named_one(target, std::forward<Actions>(An)), ...);
00909     return target;
00910 }
00911
00928 template <typename action_t, typename key_t, typename value_t, typename... Actions>
00929 requires (std::is_same_v<value_t, action_t*>)
00930 actuator<action_t> connect(std::pair<key_t, value_t> A1, Actions&&... An) {
00931     actuator<action_t> target;
00932     connect_named_one(target, std::move(A1));
00933     (connect_named_one(target, std::forward<Actions>(An)), ...);
00934     return target;
00935 }
00936
00937 // generic helpers to remove const qualifier from a function type,
00938 // for instance const member functions
00939 template <typename T>
00940 struct function_remove_const;
00941
00942 template <typename R, typename... Args>
00943 struct function_remove_const<R(Args...)> {
00944     using type = R(Args...);
00945 };
00946
00947 template <typename R, typename... Args>
00948 struct function_remove_const<R(Args...) const> {
00949     using type = R(Args...);
00950 };
00951
00955
00977 template <typename class_t, typename T,
00978     typename action_t = std::function<typename function_remove_const<T>::type>
00979 action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method) {
00980     return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) ->
00981         typename action_t::result_type {
00982             // lock() also keeps the object alive for the duration of the call
00983             const auto obj_ = wp.lock();
00984             if (!obj_) {
00985                 // inform the actuator about dead binding
00986                 throw invalid_action("bind: invalid object");
00987             }
00988             return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
00989         };
00990 }
00991
01012 template <typename class_t, typename T,
01013     typename action_t = std::function<typename function_remove_const<T>::type>
01014 action_t bind(class_t* obj, T class_t::* method) {
01015     return [obj, method](auto&&... args) -> typename action_t::result_type {
01016         if (!obj) {

```

```

01017     // inform the actuator about dead binding
01018     throw invalid_action("bind: invalid object");
01019 }
01020 return ((obj)->*method)(std::forward<decltype(args)>(args)...);
01021 };
01022 }
01023
01055 template <typename action_t, typename... Args>
01056 task_t bind_task(action_t action, Args&&... args) {
01057     using result_t = typename action_t::result_type;
01058
01059     static_assert(sizeof...(Args) > 0,
01060         "bind_task: a task must be given a callback as its last argument");
01061
01062     constexpr std::size_t last = sizeof...(Args) - 1;
01063     using callback_t = std::tuple_element_t<last, std::tuple<std::decay_t<Args>...>>;
01064
01065     static_assert(task_callback_for<callback_t, result_t>,
01066         "bind_task: the last argument must be a callback taking the task's result and "
01067         "returning nothing (void()) for a task that returns nothing");
01068
01069     // References into the caller's arguments. What is copied out of it is the task's own, below.
01070     auto pack = std::forward_as_tuple(std::forward<Args>(args)...);
01071
01072     // A task that could not do its job is built empty, and add_task() refuses it on the caller's
01073     // own stack. Only a callable that can be tested is tested: a bare lambda is never empty.
01074     if constexpr (requires { static_cast<bool>(std::get<last>(pack)); }) {
01075         if (!static_cast<bool>(std::get<last>(pack))) {
01076             return {};
01077         }
01078     }
01079     if constexpr (requires { static_cast<bool>(action); }) {
01080         if (!static_cast<bool>(action)) {
01081             return {};
01082         }
01083     }
01084
01085     // The pack is indexed rather than unpacked, because everything but its last element is bound and
01086     // a fold cannot say "all but the last".
01087     return [&<std::size_t... i>(std::index_sequence<i...>) -> task_t {
01088         return [action = std::move(action), callback = callback_t(std::get<last>(pack)),
01089             ... bound = std::decay_t<decltype(std::get<i>(pack))>(
01090                 std::get<i>(pack))]() mutable -> void {
01091             if constexpr (std::is_void_v<result_t>) {
01092                 action(bound...);
01093                 callback();
01094             } else {
01095                 // One expression, so the result is handed over as an rvalue: a result type that cannot be
01096                 // copied is still a result a task can report.
01097                 callback(action(bound...));
01098             }
01099         };
01100     }(std::make_index_sequence<last>{});
01101 }
01102
01103 } // namespace untangle

```

# Index

actions\_t  
    untangle::actuator< action\_t >, [32](#)  
actuator, [1](#)  
    untangle::actuator< action\_t >, [33](#)  
add  
    untangle::actuator< action\_t >, [37](#), [38](#), [39](#)  
add\_task  
    untangle::actuator< action\_t >, [39](#)  
  
bind  
    namespace untangle: functions, [18](#), [19](#)  
bind\_task  
    untangle, [24](#)  
  
call\_tasks  
    untangle::actuator< action\_t >, [37](#)  
connect  
    namespace untangle: functions, [15](#), [16](#), [17](#), [18](#)  
connect\_named\_one  
    untangle, [24](#)  
connect\_one  
    untangle, [23](#)  
copy\_from  
    untangle::actuator< action\_t >, [34](#)  
  
errors\_t  
    untangle::actuator< action\_t >, [32](#)  
  
has\_action  
    untangle::actuator< action\_t >, [41](#)  
has\_tasks  
    untangle::actuator< action\_t >, [41](#)  
  
invalid\_action  
    untangle::invalid\_action, [43](#)  
invoke\_action  
    untangle::actuator< action\_t >, [37](#)  
invoke\_callback  
    untangle::actuator< action\_t >, [35](#)  
is\_connected  
    untangle::actuator< action\_t >, [41](#)  
  
last\_arg  
    untangle, [23](#)  
  
namespace untangle: functions, [15](#)  
    bind, [18](#), [19](#)  
    connect, [15](#), [16](#), [17](#), [18](#)  
  
operator()  
    untangle::actuator< action\_t >, [36](#)  
  
operator=  
    untangle::actuator< action\_t >, [33](#)  
owned  
    untangle::actuator< action\_t >, [42](#)  
  
release\_owned  
    untangle::actuator< action\_t >, [34](#)  
remove  
    untangle::actuator< action\_t >, [40](#)  
reset  
    untangle::actuator< action\_t >, [34](#)  
results\_t  
    untangle::actuator< action\_t >, [32](#)  
  
task\_t  
    untangle, [22](#)  
tasks  
    untangle::actuator< action\_t >, [42](#)  
tasks\_t  
    untangle::actuator< action\_t >, [32](#)  
type  
    untangle::actuator< action\_t >, [33](#)  
  
untangle, [21](#)  
    bind\_task, [24](#)  
    connect\_named\_one, [24](#)  
    connect\_one, [23](#)  
    last\_arg, [23](#)  
    task\_t, [22](#)  
untangle::actuator< action\_t >, [29](#)  
    actions\_t, [32](#)  
    actuator, [33](#)  
    add, [37](#), [38](#), [39](#)  
    add\_task, [39](#)  
    call\_tasks, [37](#)  
    copy\_from, [34](#)  
    errors\_t, [32](#)  
    has\_action, [41](#)  
    has\_tasks, [41](#)  
    invoke\_action, [37](#)  
    invoke\_callback, [35](#)  
    is\_connected, [41](#)  
    operator(), [36](#)  
    operator=, [33](#)  
    owned, [42](#)  
    release\_owned, [34](#)  
    remove, [40](#)  
    reset, [34](#)  
    results\_t, [32](#)  
    tasks, [42](#)

- tasks\_t, [32](#)
- type, [33](#)
- untangle::invalid\_action, [42](#)
  - invalid\_action, [43](#)
  - what, [43](#)
- untangle::task\_callback\_for, [27](#)
- what
  - untangle::invalid\_action, [43](#)